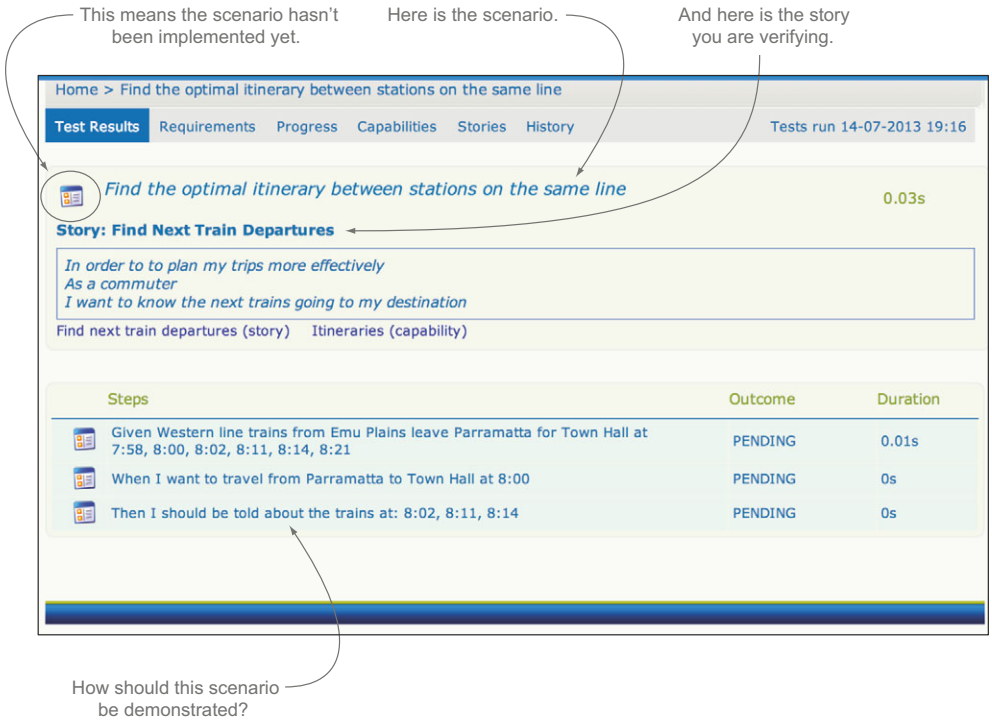


Figure 2.4 The JBehave story in the acceptance test reports

At this point, your scenario is no longer a simple text document; it's now an *executable specification*. It can be run as part of the automated build process to automatically determine whether a particular feature has been completed or not. When tests like this are first executed, they're flagged as "pending," which means, in BDD terms, that the test has been automated but that the code that implements the supporting features has not yet been written. As the features are implemented and the acceptance tests succeed, they're marked as "passed" to indicate that you've completed work in this area.

But living documentation is more than just test reporting. It should also report on the state of all of your specified requirements, even the ones that don't have any tests yet. This gives a much more complete picture of your project and your product. You can see an example of this type of requirements-level reporting by clicking on the Requirements tab in the reports you just generated (see figure 2.5). Here you should see a list of all the high-level requirements we discussed earlier, along with an idea of how much work has been completed for each of them (which, at this stage, will be precisely none).

Before we look at how to automate scenarios like this in Java, let's look at another variation. An important part of this application is the ability to tell commuters when they'll arrive at their destination if they leave at a certain time. Jill provided some examples that you can use to build up a scenario describing this requirement.